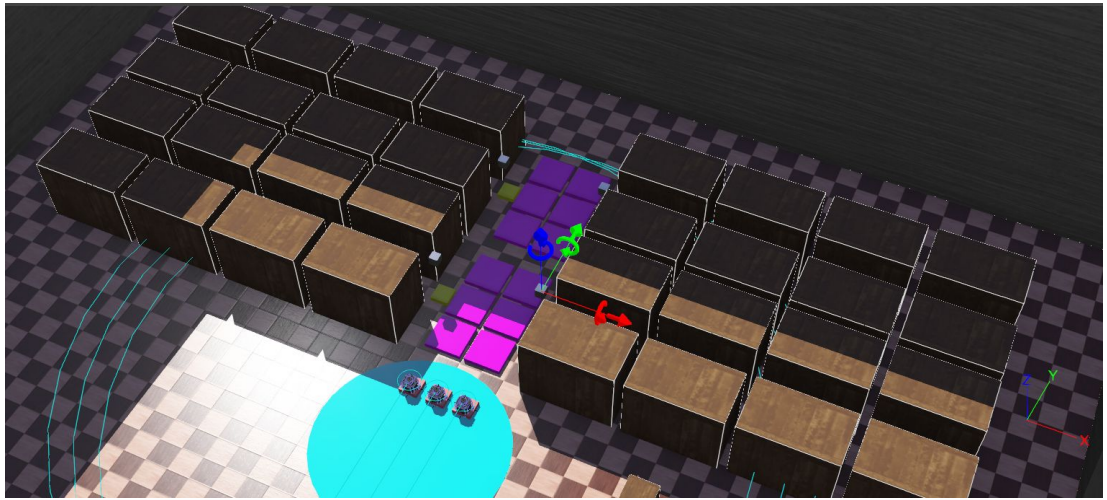
Software Engineering for Cyberphysical Systems

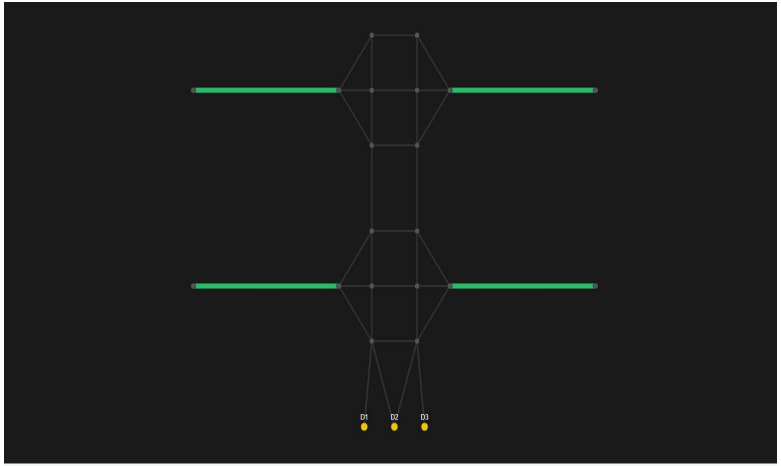
Final presentation

Florian Salaets
Lars Mertens
Tobe Nysmans

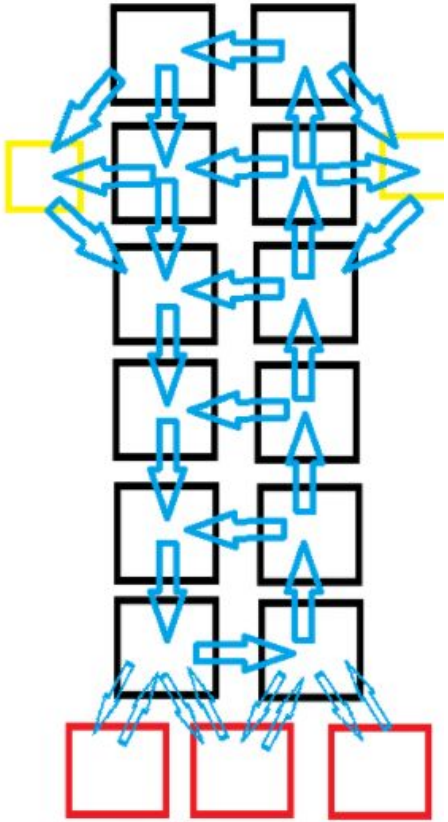
Warehouse architecure

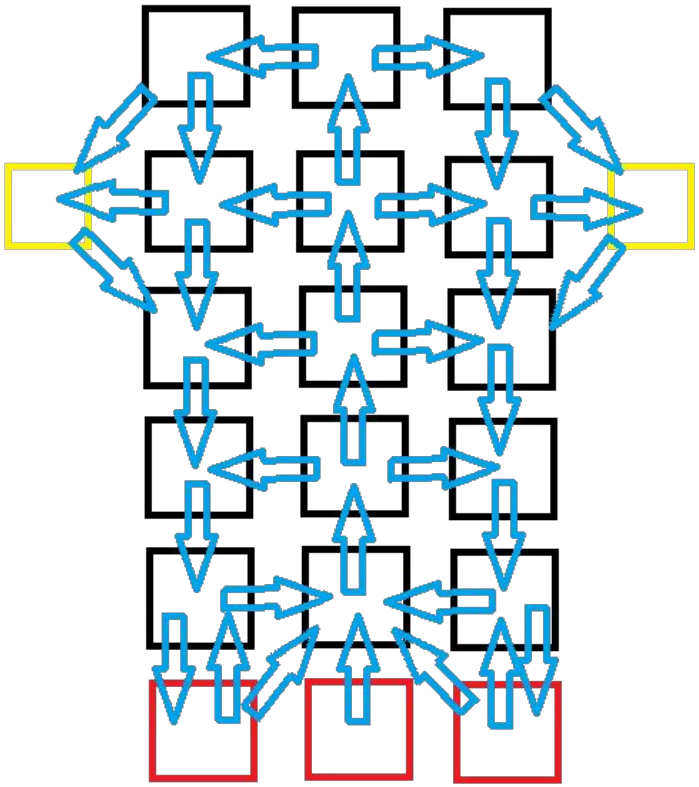


-Highway nodes
-Entrance nodes
-Aisle nodes
-Aisle devices



Highway node navigation





Path planning:

- BFS
- Non-directional edge
- expandable

Moving & rotating:

- compass & gps
- Self correcting

UHASSELT KU LEUVEN

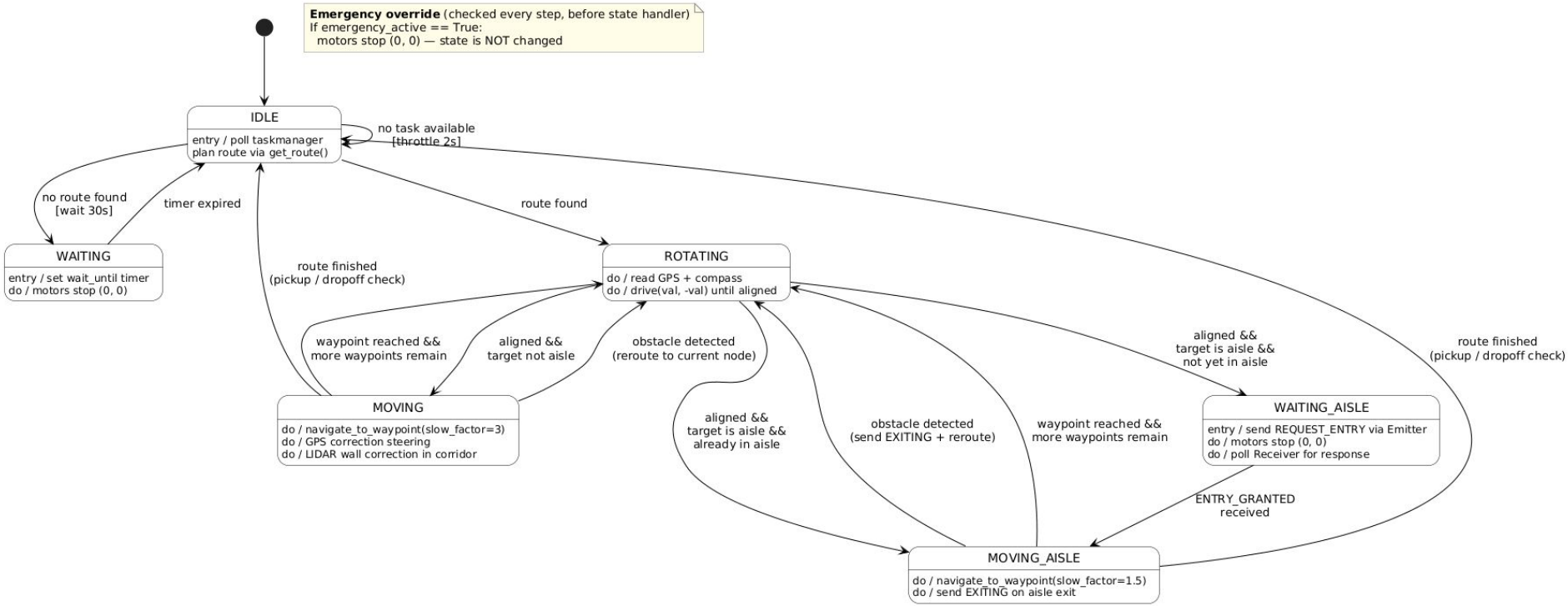
The bots

- is an extension of turtlebot Burger bot
- GPS
- Lidar
- Compass
- emitter and receiver

- Is an FSM
  - idle
  - moving
  - waiting
  - rotating
  - moving aisle
  - waiting aisle

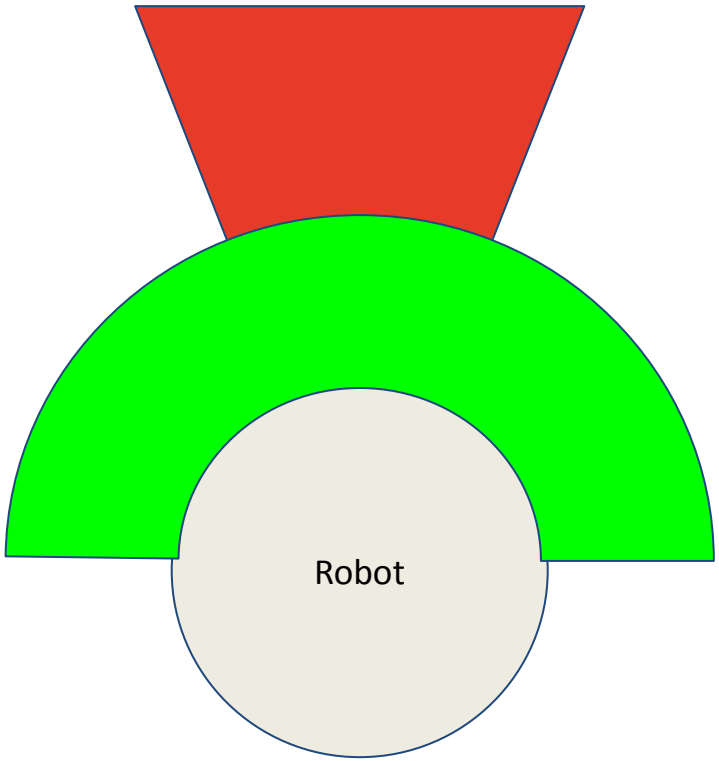
UHASSELT KU LEUVEN

FSM


Emergency override (checked every step, before state handler)
If emergency_active == True:
motors stop (0, 0) — state is NOT changed
IDLE
entry / poll taskmanager
plan route via get_route()
no task available
[throttle 2s]
no route found
[wait 30s]
timer expired
route found
WAITING
entry / set wait_until timer
do / motors stop (0, 0)
route finished
(pickup / dropoff check)
ROTATING
do / read GPS + compass
do / drive(val, -val) until aligned
waypoint reached &&
more waypoints remain
aligned &&
target not aisle
obstacle detected
(reroute to current node)
aligned &&
target is aisle &&
not yet in aisle
route finished
(pickup / dropoff check)
MOVING
do / navigate_to_waypoint(slow_factor=3)
do / GPS correction steering
do / LIDAR wall correction in corridor
aligned &&
target is aisle &&
already in aisle
obstacle detected
(send EXITING + reroute)
waypoint reached &&
more waypoints remain
WAITING_AISLE
entry / send REQUEST_ENTRY via Emitter
do / motors stop (0, 0)
do / poll Receiver for response
ENTRY_GRANTED
received
MOVING_AISLE
do / navigate_to_waypoint(slow_factor=1.5)
do / send EXITING on aisle exit

Collision prevention

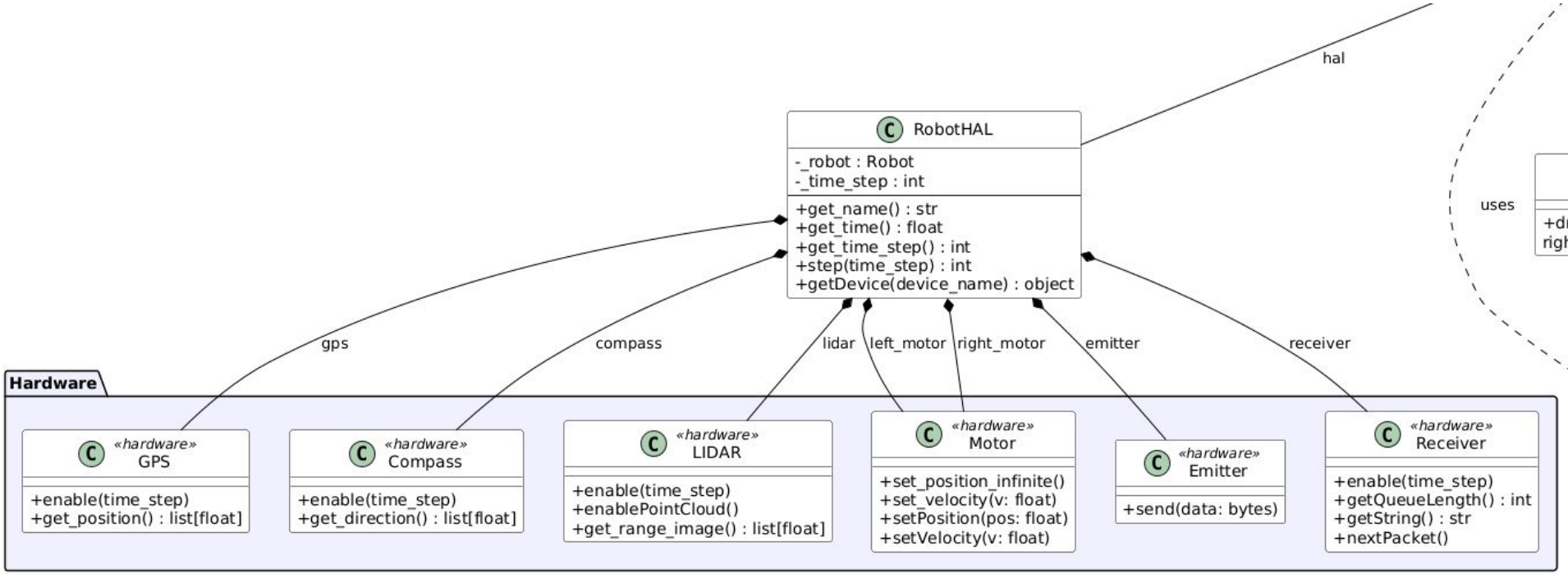
- Lidar
- detection systems
- Path obstructed
- slowdown
- Aisle centering



HAL

- Hardware Abstraction Layer
- portability
- Hardware independence
- 3 part HAL
- Harware components
- robot HAL
- aisleDevice HAl

HAL


RobotHAL
-_robot : Robot
-_time_step : int
+get_name() : str
+get_time() : float
+get_time_step() : int
+step(time_step) : int
+getDevice(device_name) : object
hal
uses
gps
compass
lidar
left_motor
right_motor
emitter
receiver
Hardware
«hardware»
GPS
+enable(time_step)
+get_position() : list[float]
«hardware»
Compass
+enable(time_step)
+get_direction() : list[float]
«hardware»
LIDAR
+enable(time_step)
+enablePointCloud()
+get_range_image() : list[float]
«hardware»
Motor
+set_position_infinite()
+set_velocity(v: float)
+setPosition(pos: float)
+setVelocity(v: float)
«hardware»
Emitter
+send(data: bytes)
«hardware»
Receiver
+enable(time_step)
+getQueueLength() : int
+getString() : str
+nextPacket()

Aisle navigation + locking

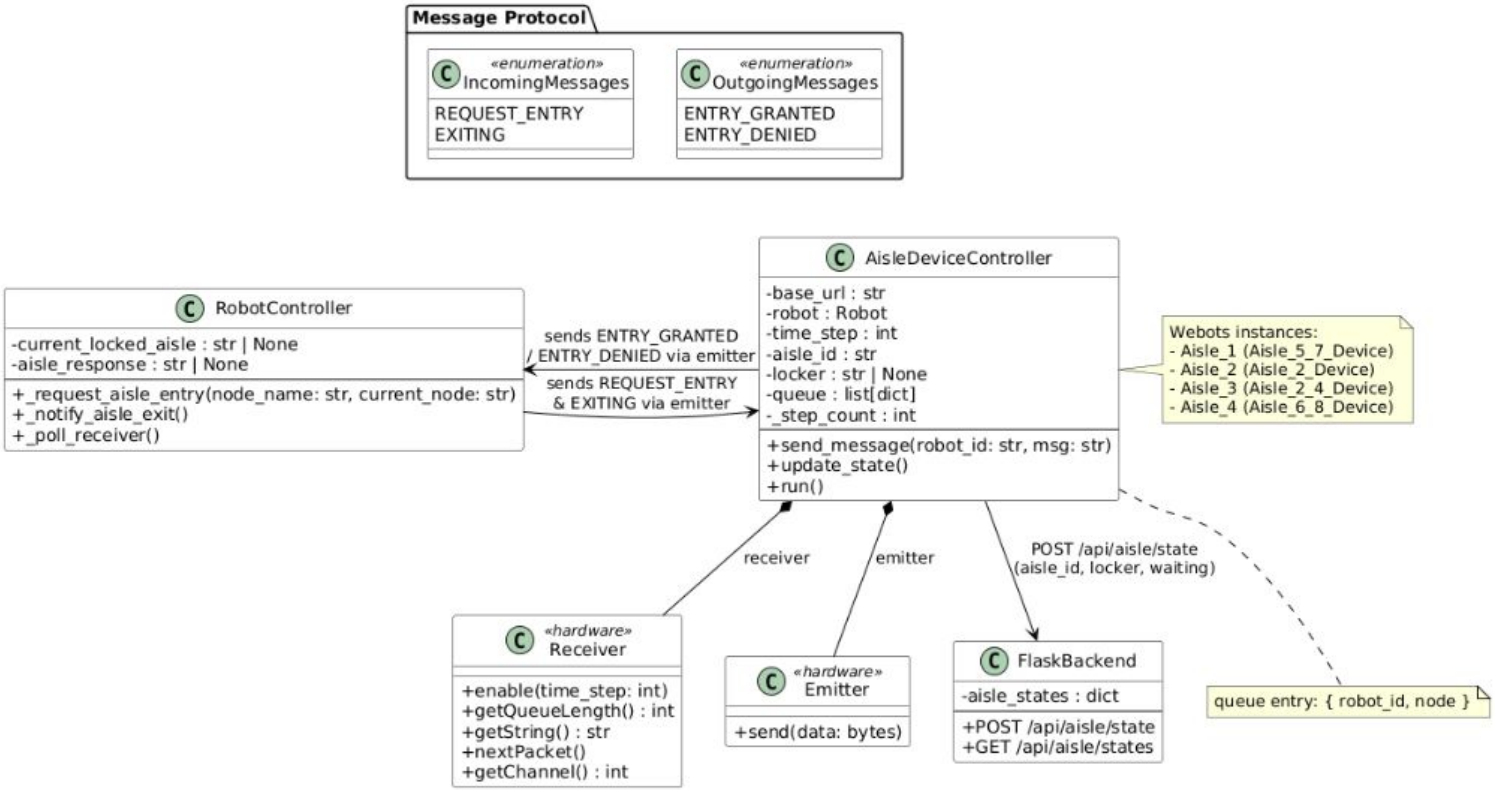
-Navigation

- Lidar
- bidirectional edge

-Locking

- aisle Device

Aisle device



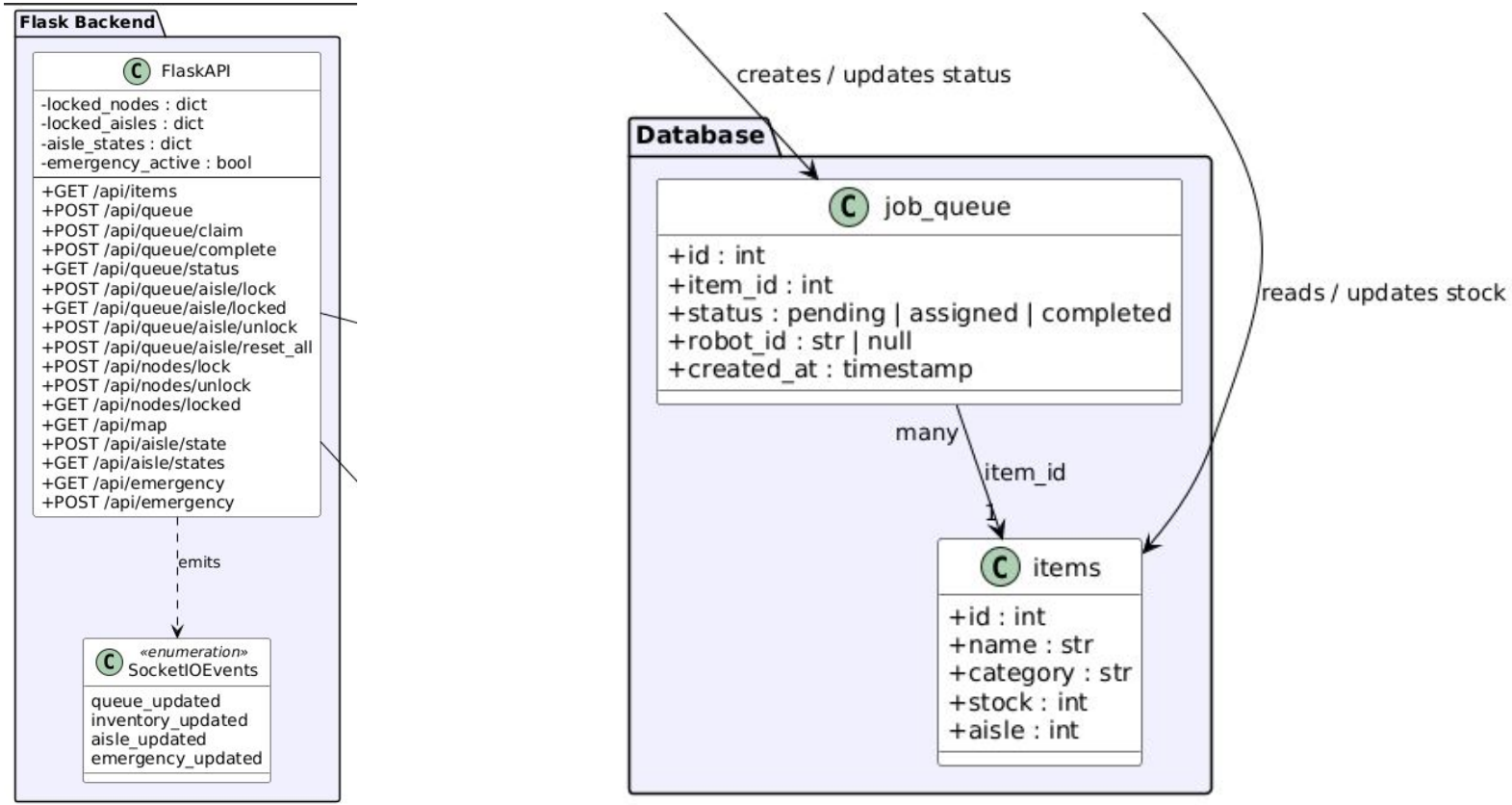
Backend

-Flask server for all API endpoints
-Uses Websockets to communicate with the frontend
-is the connection between the frontend and the bots
- Handles all the SQL queries to the DB

- concurrent tasks

UHASSELT KU LEUVEN

Backend


Flask Backend
FlaskAPI
-locked_nodes : dict
-locked_aisles : dict
-aisle_states : dict
-emergency_active : bool
+GET /api/items
+POST /api/queue
+POST /api/queue/claim
+POST /api/queue/complete
+GET /api/queue/status
+POST /api/queue/aisle/lock
+GET /api/queue/aisle/locked
+POST /api/queue/aisle/unlock
+POST /api/queue/aisle/reset_all
+POST /api/nodes/lock
+POST /api/nodes/unlock
+GET /api/nodes/locked
+GET /api/map
+POST /api/aisle/state
+GET /api/aisle/states
+GET /api/emergency
+POST /api/emergency
emits
«enumeration»
SocketIOEvents
queue_updated
inventory_updated
aisle_updated
emergency_updated
creates / updates status
Database
job_queue
+id : int
+item_id : int
+status : pending | assigned | completed
+robot_id : str | null
+created_at : timestamp
reads / updates stock
many
item_id
1
items
+id : int
+name : str
+category : str
+stock : int
+aisle : int

Database

-SQL database
- 2 Tables
  - Items
  - Qeue
- atomic updates for task dividing

UHASSELT KU LEUVEN

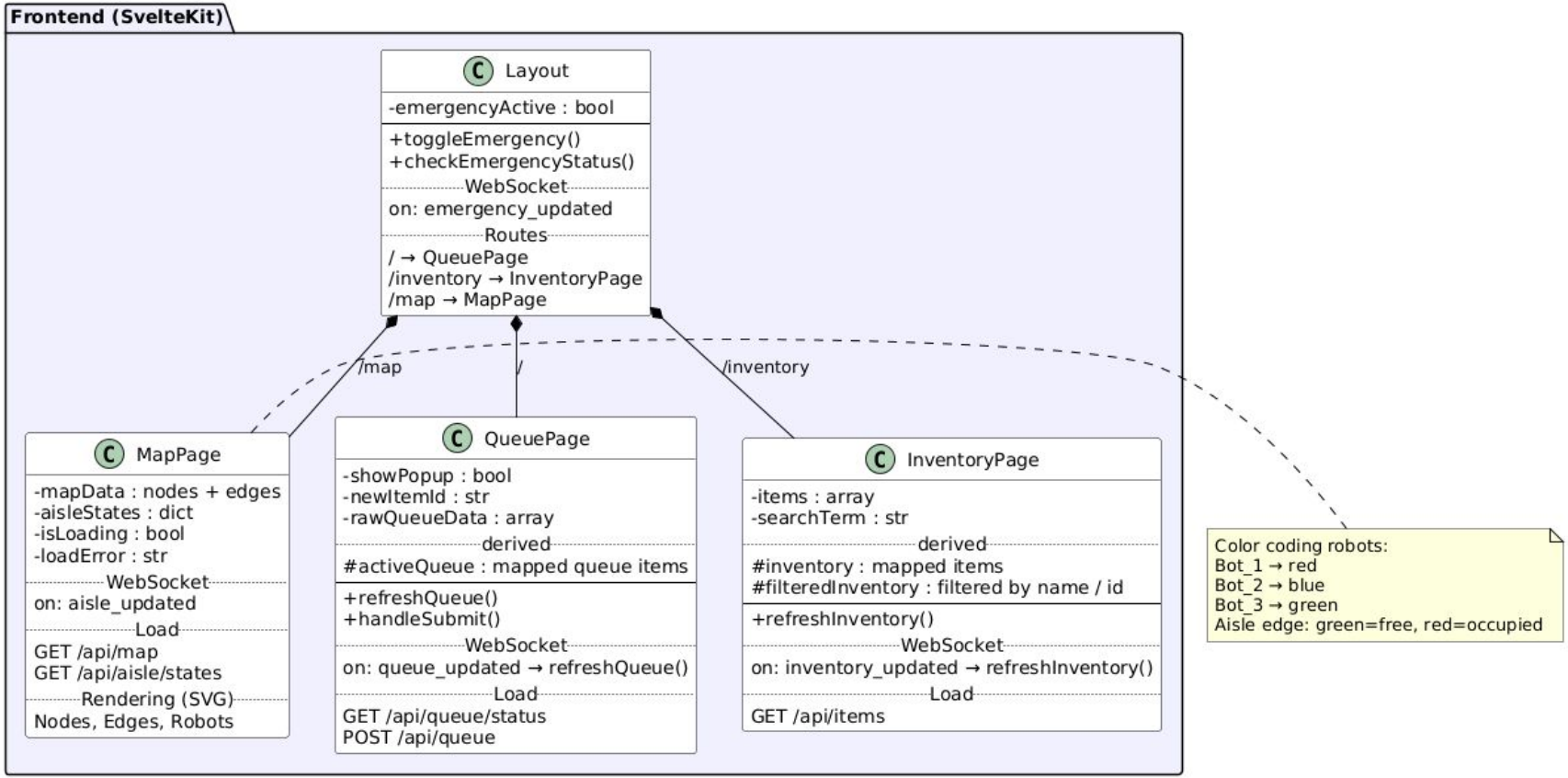
Frontend

-Svelte
-3 pages

1. Live qeue
2. Inventory
3. Warehouse Map

- Emergency button for stopping all movement

Frontend

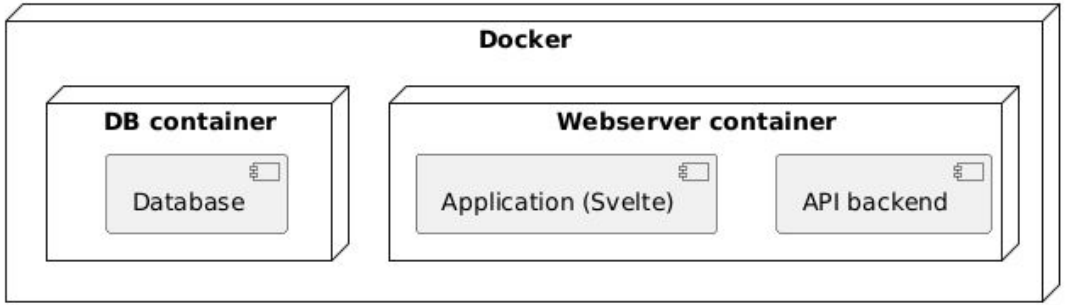

Frontend (SvelteKit)
Layout
-emergencyActive : bool
+toggleEmergency()
+checkEmergencyStatus()
WebSocket
on: emergency_updated
Routes
/ → QueuePage
/inventory → InventoryPage
/map → MapPage
/map
/
/inventory
MapPage
-mapData : nodes + edges
-aisleStates : dict
-isLoading : bool
-loadError : str
WebSocket
on: aisle_updated
Load
GET /api/map
GET /api/aisle/states
Rendering (SVG)
Nodes, Edges, Robots
QueuePage
-showPopup : bool
-newItemId : str
-rawQueueData : array
derived
#activeQueue : mapped queue items
+refreshQueue()
+handleSubmit()
WebSocket
on: queue_updated → refreshQueue()
Load
GET /api/queue/status
POST /api/queue
InventoryPage
-items : array
-searchTerm : str
derived
#inventory : mapped items
#filteredInventory : filtered by name / id
+refreshInventory()
WebSocket
on: inventory_updated → refreshInventory()
Load
GET /api/items
Color coding robots:
Bot_1 → red
Bot_2 → blue
Bot_3 → green
Aisle edge: green=free, red=occupied

Concurrency

- Using node system
- preventing collisions
- rerouting when necessary
- 1 task per robot
- single source of truth (DB)

Deployment

- Docker
- **Frontend:** localhost::5173
- **Backend:** localhost::5000
- **Database**
- Volumes:
  - MAP
  - initial database

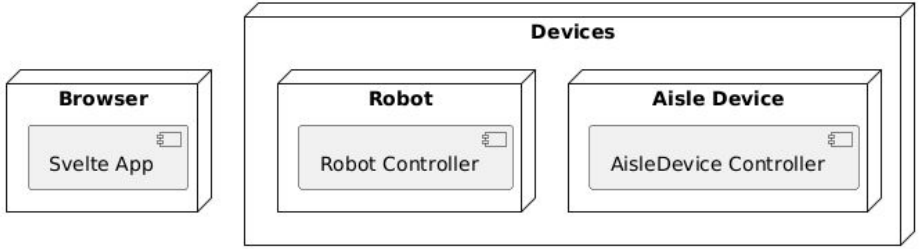



```yaml
services:
  db:
    image: mysql:8.0
    container_name: secs-db
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: warehouse
      MYSQL_USER: secs
      MYSQL_PASSWORD: secs
    ports:
      - "3306:3306"
    volumes:
      - db_data:/var/lib/mysql
      - ./database/init.sql:/docker-entrypoint-initdb.d/01-init.sql:ro
    command: --default-authentication-plugin=mysql_native_password

  backend:
    build: ./backend
    container_name: secs-backend
    restart: always
    ports:
      - "5000:5000"
    environment:
      DB_HOST: db  # In Docker gebruiken we de servicenaam als host!
      DB_USER: secs
      DB_PASSWORD: secs
      DB_NAME: warehouse
    volumes:
      - ./controllers/robot_controller/map.json:/app/controllers/robot_controller/map.json
    depends_on:
      - db

  frontend:
    build: ./Frontend/robot-dashboard
    container_name: secs-frontend
    ports:
      - "5173:3000"
    depends_on:
      - backend

volumes:
  db_data:
```

Deployment

- **Browser** used to show frontend

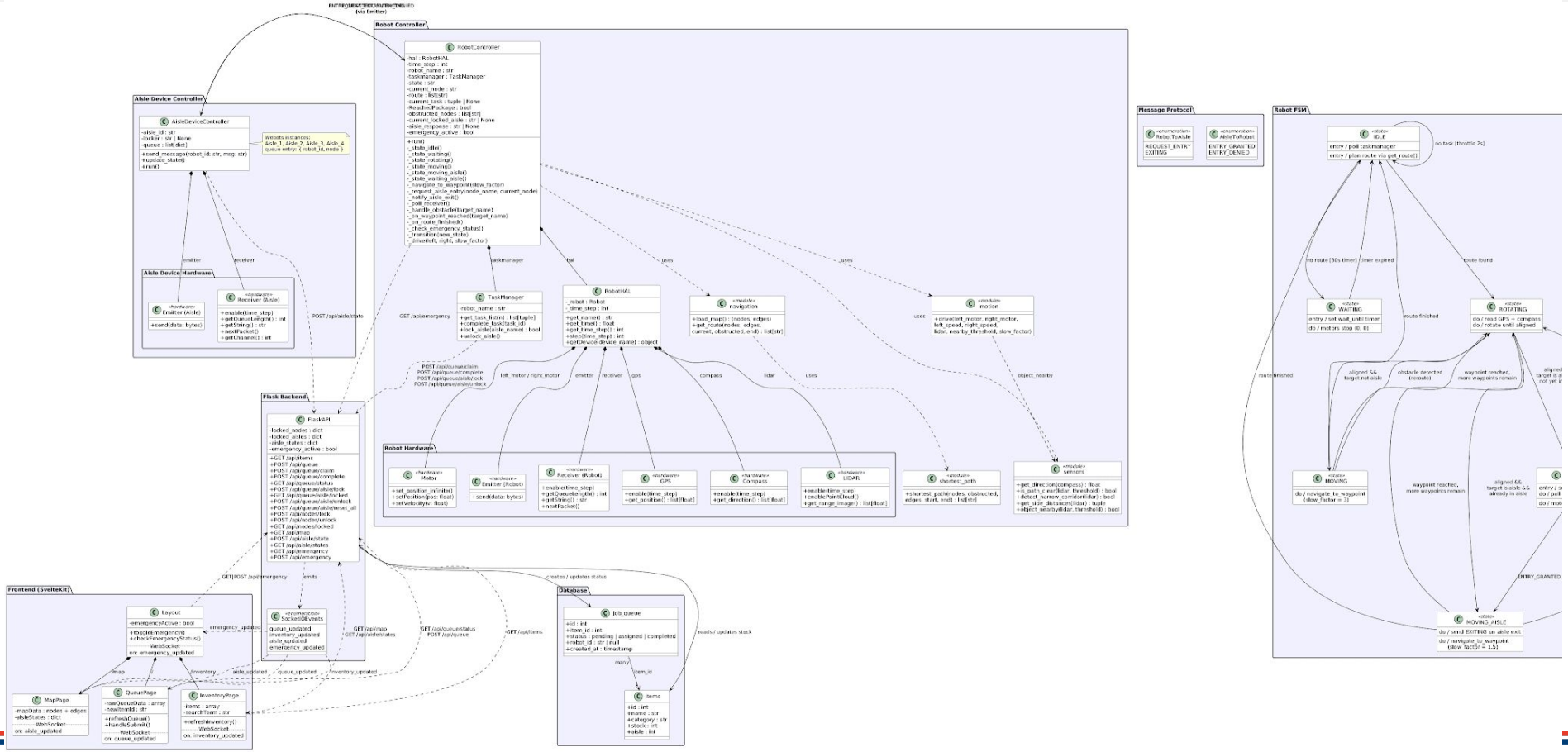
- **Devices**
  - **Robot**
    - HAL integration
    - mobile actor
  - **Aisle device**
    - HAL wrapper
    - traffic controller for aisles
    - aisle status



Testing

-shortest path was tested with written test
-mostl other tests trough webots
-using dummy data/bots for edge cases
-Debug messages

Q&A


Robot Controller
Aisle Device Controller
Aisle Device Hardware
Flask Backend
Robot Hardware
Frontend (SvelteKit)
Database
Message Protocol
Robot FSM
RobotController
AisleDeviceController
Emitter (Aisle)
Receiver (Aisle)
TaskManager
RobotHAL
navigation
motion
FlaskAPI
Mobv
Emitter (Robot)
Receiver (Robot)
GPS
Compass
LIDAR
shortest_path
sensors
Layout
MapPage
QueuePage
InventoryPage
job_queue
items
RobotToAisle
AisleToRobot
IDLE
WAITING
ROTATING
MOVING
MOVING_AISLE